Stochastic Gradient Descent

Lecture 25

Dr. Colin Rundel

Gradient Descent for Regression

A regression example

```r
set.seed(1234)
n = 200; p = 20
X = matrix(rnorm(n * p), n, p)
true_beta = rep(0, p)
true_beta[c(3, 7, 12, 18)] = c(5.2, -3.1, 8.7, -1.5)
y = 3 + X %*% true_beta + rnorm(n)
```

```r
head(y, 20)
```

```
             [,1]
 [1,]  4.54630945
 [2,] -0.27563573
 [3,] -8.92201672
 [4,]  3.81019348
 [5,] -8.01772803
 [6,] -5.65360256
 [7,] -9.45140764
 [8,]  4.92135279
 [9,] -4.13507430
[10,]  6.77822004
[11,] -1.16028786
[12,] 20.83784033
[13,]  4.07084356
[14,]  9.58649951
[15,]  1.41834659
[16,] 13.08886089
[17,]  8.68524259
[18,] -1.72856652
[19,]  0.97934767
[20,]  0.03342649
```

```r
true_beta
```

```
 [1]  0.0  0.0  5.2  0.0  0.0  0.0 -3.1  0.0  0.0  0.0  0.0  8.
[14]  0.0  0.0  0.0  0.0 -1.5  0.0  0.0
```

Minimalistic GD for linear regression

```
grad_desc_lm = function(X, y, beta, step, max_step = 50) {
  X = cbind(1, X)
  n = nrow(X); k = ncol(X)

  f = \(beta) sum((y - X %*% beta)^2)
  grad = \(beta) 2 * t(X) %*% (X %*% beta - y)

  res = list(x = list(beta), loss = f(beta), iter = 0)

  for (i in seq_len(max_step)) {
    beta = beta - drop(grad(beta)) * step
    res$x = c(res$x, list(beta))
    res$loss = c(res$loss, f(beta))
    res$iter = c(res$iter, i)
  }

  res
}
```

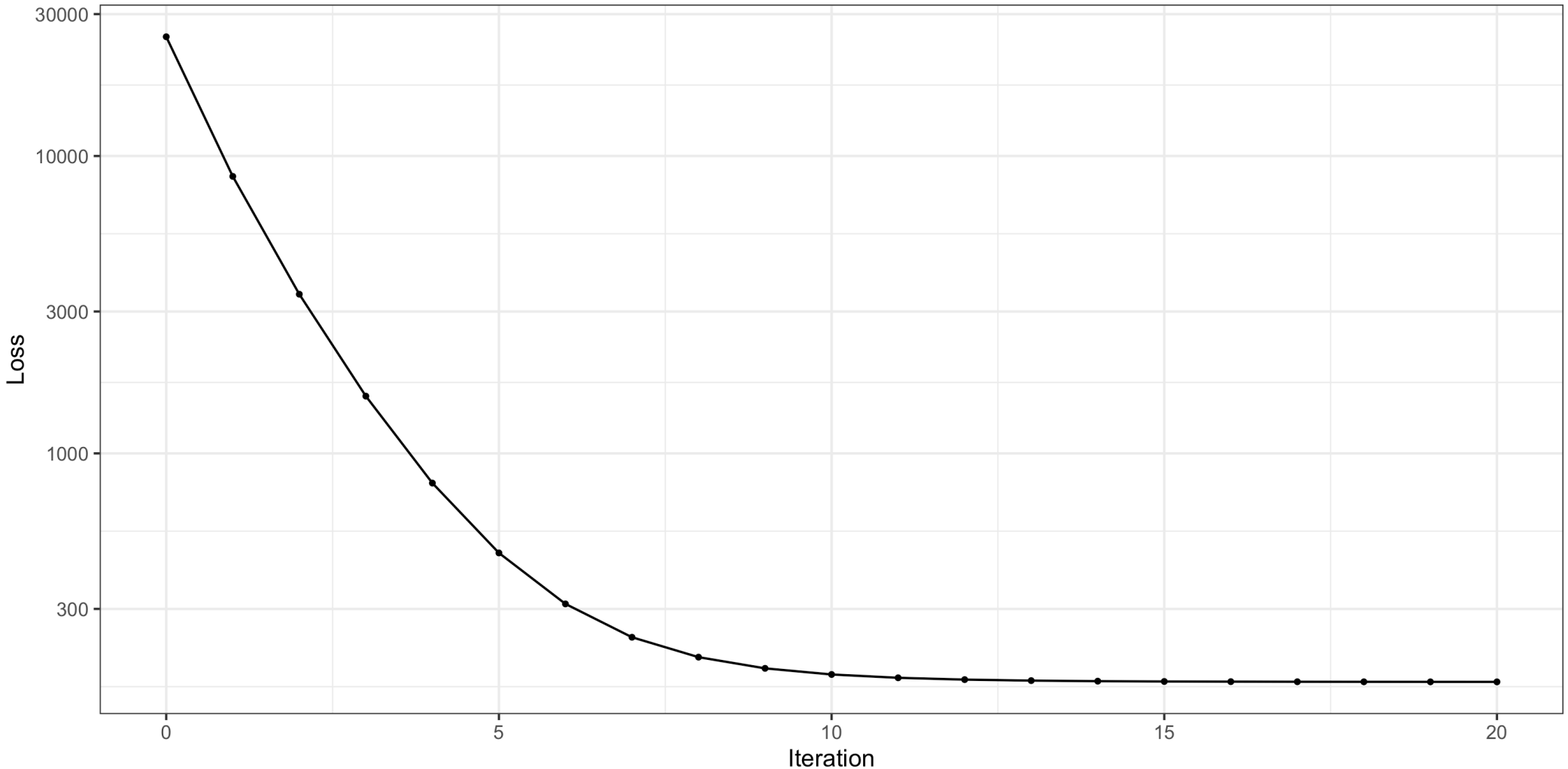
Linear regression

```
lm_fit = lm(y ~ X)
coef(lm_fit)
```

```
 (Intercept)           X1           X2           X3           X4
 3.117181706 -0.072830269  0.008036239  5.342001598  0.048250726
          X5           X6           X7           X8           X9
 0.073180263  0.020204905 -3.070971288 -0.044335042  0.100285902
         X10          X11          X12          X13          X14
 0.039388406  0.098979483  8.655554445 -0.026946984  0.096013967
         X15          X16          X17          X18          X19
-0.007404586  0.114183408 -0.003962137 -1.437613569  0.074029481
         X20
-0.064314202
```

```
gd_time = system.time({
  gd_lm = grad_desc_lm(
    X, y, rep(0, p + 1),
    step = 0.001, max_step = 20
  )
})
tail(gd_lm$x, 1)[[1]]
```

```
 [1]  3.114903770 -0.080977020  0.007570766  5.338037675  0.046141346
 [6]  0.067510425  0.019478840 -3.064915138 -0.038321282  0.103133275
[11]  0.040073545  0.100464352  8.651927265 -0.019456634  0.092119411
```


30000
10000
3000
1000
300
Loss
0
5
10
15
20
Iteration

A quick analysis

Let's take a quick look at the linear regression loss function and gradient descent and think a bit about its cost(s), we can define the loss function and its gradient as follows:

[Math Processing Error]

What are the costs of calculating the loss function and gradient respectively in terms of n and k?

- Calculating the loss function costs $O(nk)$
- Calculating the gradient costs $O(nk)$

Stochastic Gradient Descent

Stochastic Gradient Descent

This is a variant of gradient descent where rather than using all n data points we randomly sample one at a time and use that single point to make our gradient step.

- Sampling of observations can be done with or without replacement
- Will take more steps to converge but each step is now cheaper to compute
- SGD has slower asymptotic convergence than GD, but is often faster in practice in terms of runtime
- Generally requires the learning rate to shrink as a function of iteration to guarantee convergence

SGD - Linear Regression

```r
sgd_lm = function(X, y, beta, step, max_step = 50, seed = 1234, replace = TRUE) {
  X = cbind(1, X)
  n = nrow(X); k = ncol(X)
  f = \(beta) sum((y - X %*% beta)^2)
  grad_i = \(beta, i) 2 * X[i, ] * drop(X[i, ] %*% beta - y[i])

  res = list(x = list(beta), loss = f(beta), iter = 0)
  set.seed(seed)

  for (ep in seq_len(max_step)) {
    if (replace) {
      js = sample.int(n, n, replace = TRUE)
    } else {
      js = sample.int(n)
    }

    for (j in js) {
      beta = beta - grad_i(beta, j) * step
      res$x = c(res$x, list(beta))
      res$loss = c(res$loss, f(beta))
      res$iter = c(res$iter, tail(res$iter, 1) + 1)
    }
  }
```

Fitting

```r
coef(lm_fit)
```

```
 (Intercept)           X1           X2           X3           X4
 3.117181706 -0.072830269  0.008036239  5.342001598  0.048250726
          X5           X6           X7           X8           X9
 0.073180263  0.020204905 -3.070971288 -0.044335042  0.100285902
         X10          X11          X12          X13          X14
 0.039388406  0.098979483  8.655554445 -0.026946984  0.096013967
         X15          X16          X17          X18          X19
-0.007404586  0.114183408 -0.003962137 -1.437613569  0.074029481
         X20
-0.064314202
```

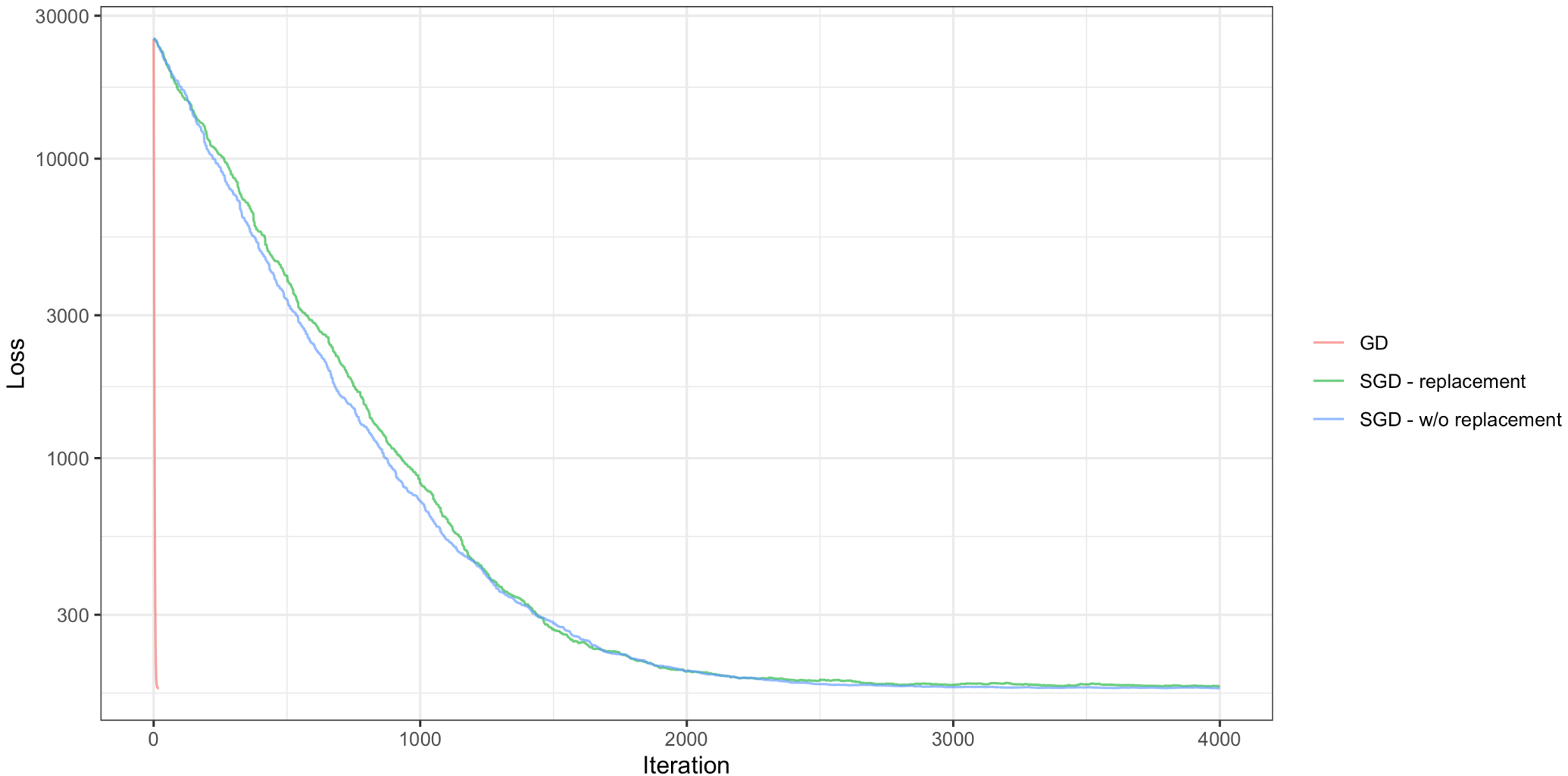
```r
sgd_rep_time = system.time({
  sgd_lm_rep = sgd_lm(
    X, y, rep(0, p + 1),
    step = 0.001, max_step = 20, replace = TRUE
  )
})
tail(sgd_lm_rep$x, 1)[[1]]
```

```
 [1]  3.1056238623 -0.0721809011  0.0008264944  5.3810608734
 [5]  0.0626057077  0.0682170030  0.0516970494 -3.0433005950
 [9] -0.0583101204  0.0879622000  0.0741407715  0.1288032144
[13]  8.6529440558  0.0060573407  0.1243799970  0.0091025772
[17]  0.1406381229 -0.0628668819 -1.4461988825  0.0316160082
[21] -0.0822805503
```

```r
sgd_worep_time = system.time({
  sgd_lm_worep = sgd_lm(
    X, y, rep(0, p + 1),
    step = 0.001, max_step = 20, replace = FALSE
  )
})
tail(sgd_lm_worep$x, 1)[[1]]
```

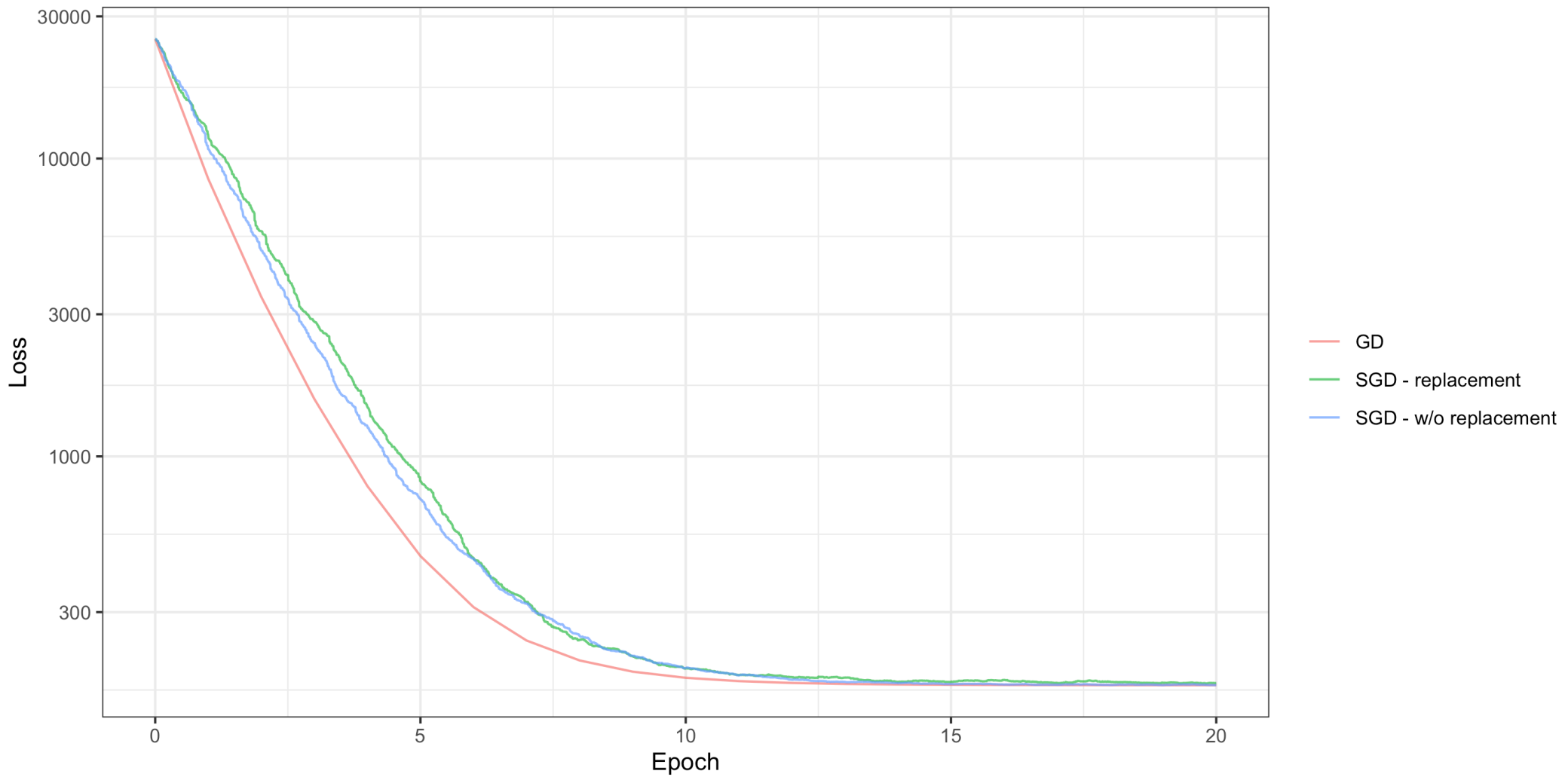
```
 [1]  3.1119100906 -0.0807910057  0.0006135997  5.3397983867
 [5]  0.0439536116  0.0684804265  0.0222715352 -3.0575691760
 [9] -0.0383532506  0.0995485898  0.0365061357  0.1059022940
[13]  8.6519504444 -0.0122412279  0.0843442651 -0.0060952897
[17]  0.1049146978 -0.0053591951 -1.4323846427  0.0702150767
[21] -0.0715025688
```

Learning by iteration



Learning by Epochs

Generally, rather than thinking in iterations we use epochs instead - an epoch is one complete pass through the data.



Run times

Method	Time / Epoch
GD	0.000s
SGD (replacement)	0.004s
SGD (w/o replacement)	0.004s

A bigger example

```r
set.seed(1234)
n = 10000; p = 20
X = matrix(rnorm(n * p), n, p)
true_beta = rep(0, p)
true_beta[c(3, 7, 12, 18)] = c(5.2, -3.1, 8.7, -1.5)
y = 3 + X %*% true_beta + rnorm(n)
```

```r
lm_fit = lm(y ~ X)
coef(lm_fit)
```

```
 (Intercept)            X1            X2            X3            X4
3.0259886890  0.0095148531 -0.0030007519  5.2180911419 -0.0064479969
          X5            X6            X7            X8            X9
-0.0045324399  0.0047502439 -3.0998533849 -0.0073794151  0.0016343803
         X10           X11           X12           X13           X14
-0.0067191472 -0.0067526804  8.6948694929 -0.0242690949 -0.0073358207
         X15           X16           X17           X18           X19
 0.0154229012  0.0194983967  0.0073739365 -1.4845618663 -0.0008924662
         X20
-0.0082218598
```

Fitting

```
gd_time = system.time({
  gd_lm = grad_desc_lm(
    X, y, rep(0, p + 1),
    step = 0.00005, max_step = 3
  )
})
tail(gd_lm$x, 1)[[1]]
```

```
 [1]  3.025751e+00  9.520621e-03 -2.769907e-03  5.218955e+00
 [5] -5.910811e-03 -5.040789e-03  5.796313e-03 -3.100655e+00
 [9] -7.750815e-03  1.838037e-03 -5.949647e-03 -6.690778e-03
[13]  8.695432e+00 -2.446508e-02 -7.531120e-03  1.498008e-02
[17]  1.946761e-02  6.796369e-03 -1.484612e+00 -8.383027e-05
[21] -8.508477e-03
```

```
sgd_rep_time = system.time({
  sgd_lm_rep = sgd_lm(
    X, y, rep(0, p + 1),
    step = 0.001, max_step = 3, replace = TRUE
  )
})
tail(sgd_lm_rep$x, 1)[[1]]
```

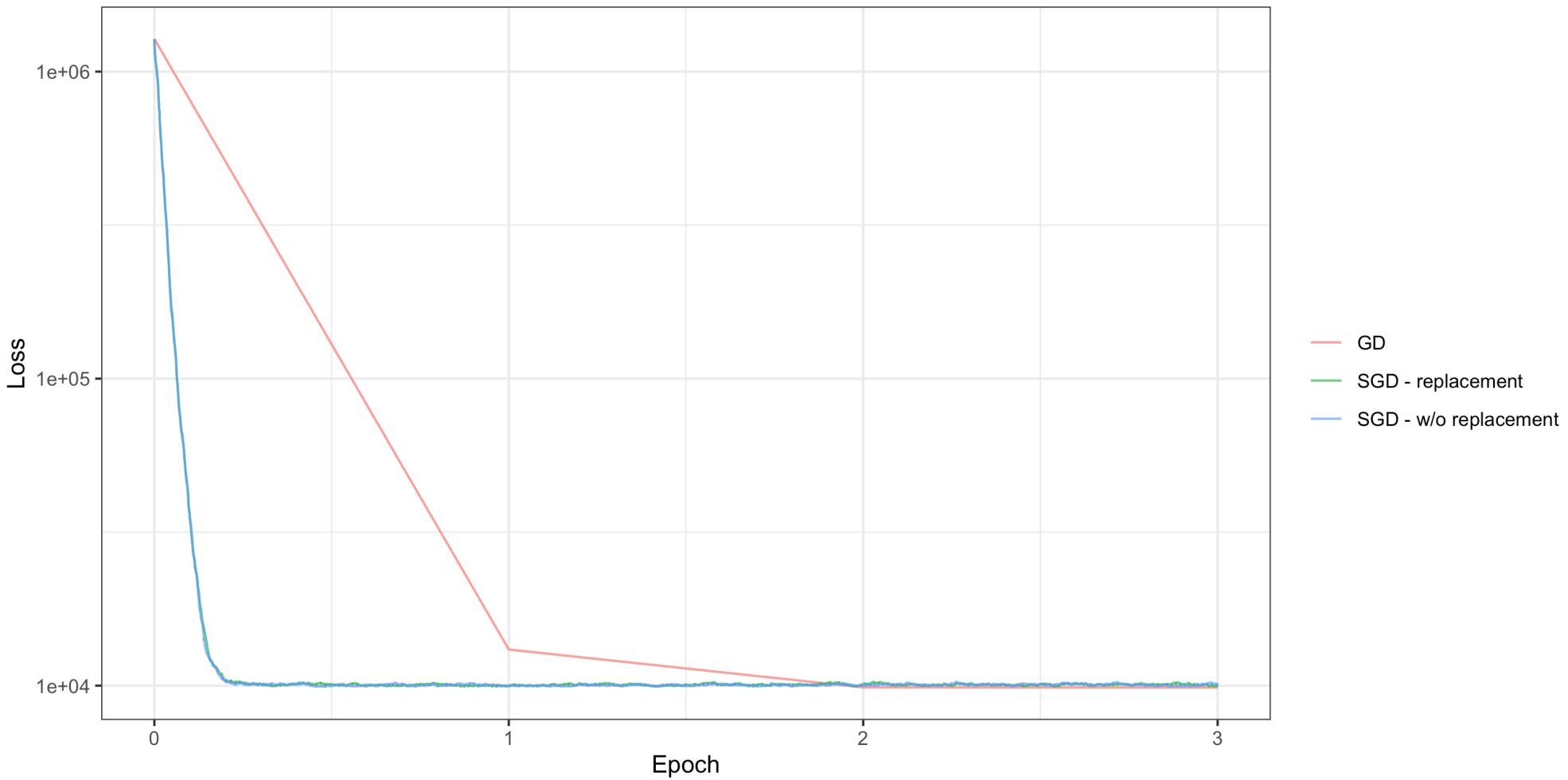
```
 [1]  3.002304748  0.003185849  0.006723989  5.240827084 -0.014486903
 [6] -0.001361589 -0.030246603 -3.140968339 -0.027647330 -0.018542458
[11] -0.013957312 -0.026383164  8.687112461 -0.011774055  0.012264836
[16]  0.061105563 -0.014438609 -0.035289160 -1.512169159  0.013478253
[21] -0.022082473
```

```
sgd_worep_time = system.time({
  sgd_lm_worep = sgd_lm(
    X, y, rep(0, p + 1),
    step = 0.001, max_step = 3, replace = FALSE
  )
})
tail(sgd_lm_worep$x, 1)[[1]]
```

```
 [1]  2.9679431569 -0.0375121215  0.0188192457  5.1526231506
 [5]  0.0321040011 -0.0214736033  0.0386542143 -3.0880526686
 [9]  0.0261735670 -0.0275259262 -0.0020874686 -0.0456127050
[13]  8.7182752251 -0.0539681021 -0.0134157549 -0.0076555352
[17]  0.0104846242  0.0008692615 -1.4688457435  0.0299891370
[21]  0.0442189766
```

Results

Full | Zoom | Timings



Mini-batch Gradient Descent

Mini-batch gradient descent

This is a variant of stochastic gradient descent where a subset of m data points is selected for each gradient update.

- The idea is to find a balance between the cost of increasing the data size vs the speed-up of vectorized calculations.
- More updates per epoch than GD, but less than SGD
- Mini batch composition can be constructed by sampling data with or without replacement

MBGD - Linear Regression

```r
mb_grad_desc_lm = function(X, y, beta, step, batch_size = 10, max_step = 50, seed = 1234, rep
  X = cbind(1, X)
  n = nrow(X); k = ncol(X)
  f = \(beta) sum((y - X %*% beta)^2)
  grad_b = \(beta, idx) 2 * t(X[idx, , drop = FALSE]) %*% (X[idx, , drop = FALSE] %*% beta -

  res = list(x = list(beta), loss = f(beta), iter = 0)
  set.seed(seed)

  for (ep in seq_len(max_step)) {
    if (replace) {
      js = sample.int(n, n, replace = TRUE)
    } else {
      js = sample.int(n)
    }

    batches = split(js, ceiling(seq_along(js) / batch_size))
    for (batch in batches) {
      beta = beta - drop(grad_b(beta, batch)) * step
      res$x = c(res$x, list(beta))
      res$loss = c(res$loss, f(beta))
      res$iter = c(res$iter, tail(res$iter, 1) + 1)
    }
```

Fitting

```r
coef(lm_fit)
```

```
  (Intercept)            X1            X2            X3            X4
 3.0259886890  0.0095148531 -0.0030007519  5.2180911419 -0.0064479969
           X5            X6            X7            X8            X9
-0.0045324399  0.0047502439 -3.0998533849 -0.0073794151  0.0016343803
          X10           X11           X12           X13           X14
-0.0067191472 -0.0067526804  8.6948694929 -0.0242690949 -0.0073358207
          X15           X16           X17           X18           X19
 0.0154229012  0.0194983967  0.0073739365 -1.4845618663 -0.0008924662
          X20
-0.0082218598
```

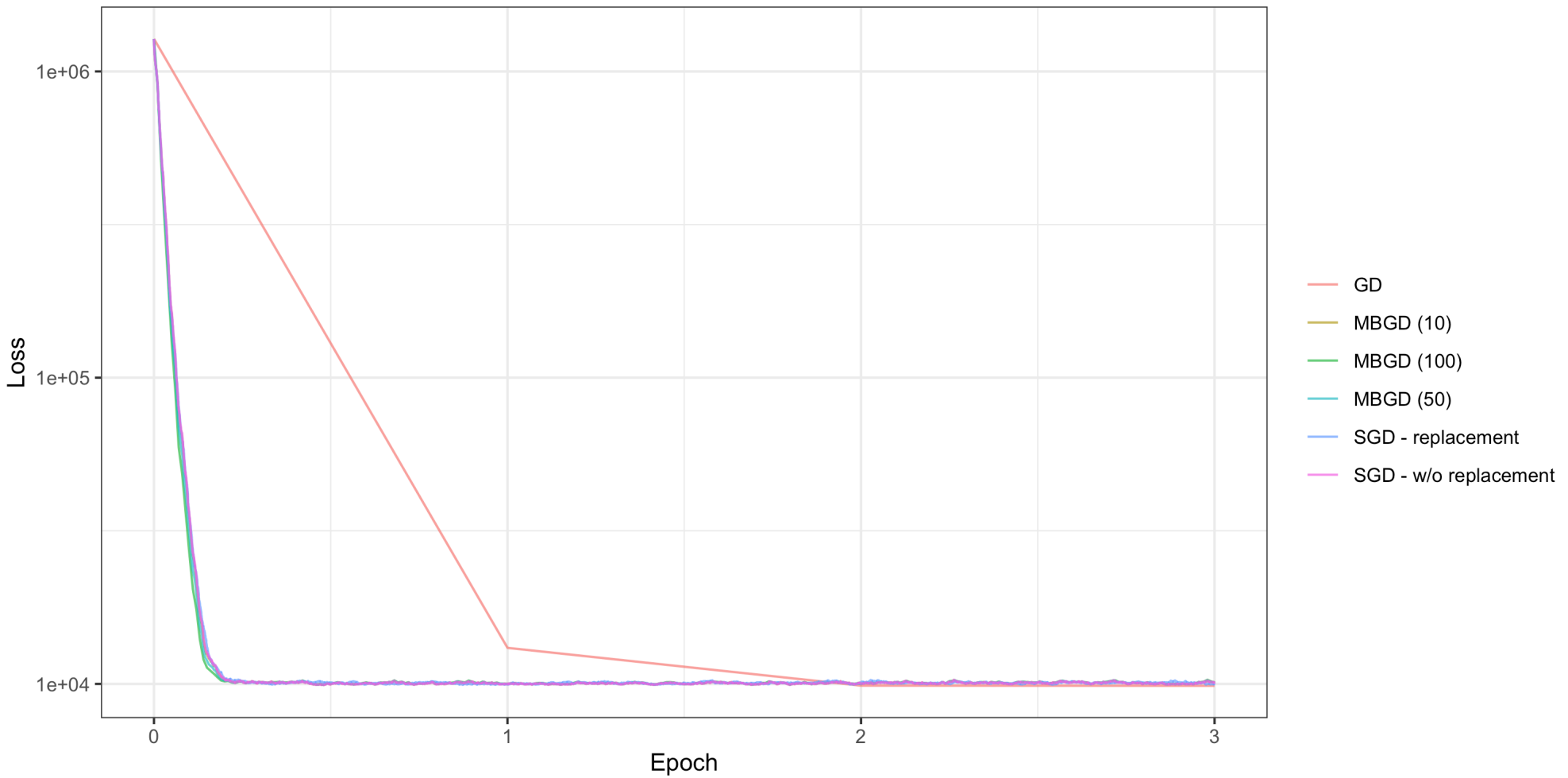
```r
sizes = c(10, 50, 100)
mbgd = list()
mbgd_time = list()
for (size in sizes) {
  mbgd_time[[as.character(size)]] = system.time({
    mbgd[[as.character(size)]] = mb_grad_desc_lm(
      X, y, rep(0, p + 1), batch_size = size,
      step = 0.001, max_step = 3, replace = FALSE
    )
  })
}
```

```
Batch size: 10 (0.125s / epoch)
 [1]  2.968354e+00 -3.776716e-02  1.917841e-02  5.152190e+00
 [5]  3.267337e-02 -2.203598e-02  3.925246e-02 -3.088205e+00
 [9]  2.556167e-02 -2.856210e-02 -1.992842e-03 -4.652826e-02
[13]  8.718959e+00 -5.308292e-02 -1.415282e-02 -7.839514e-03
[17]  9.103843e-03  3.563301e-05 -1.469968e+00  2.915876e-02
[21]  4.425822e-02

Batch size: 50 (0.026s / epoch)
 [1]  2.9674187070 -0.0394568263  0.0192840953  5.1520741377
 [5]  0.0342516304 -0.0221178931  0.0424259991 -3.0883349619
```

```
 [9]  0.0272465421 -0.0310345416 -0.0025483528 -0.0503064404
[13]  8.7184617783 -0.0541720720 -0.0139878848 -0.0093859777
[17]  0.0055787502  0.0006713781 -1.4691435321  0.0293583353
[21]  0.0468398320

Batch size: 100 (0.013s / epoch)
 [1]  2.9630757146 -0.0400186005  0.0205002683  5.1519094699
 [5]  0.0320665132 -0.0246647532  0.0445065263 -3.0879979154
 [9]  0.0269266102 -0.0325628164 -0.0022231909 -0.0540224503
[13]  8.7251980369 -0.0509923466 -0.0140196146 -0.0096701442
[17]  0.0034647604  0.0004502262 -1.4654752590  0.0304021740
[21]  0.0476232702
```

Results

Full | Zoom | Timings



A bit of theory

We've talked a bit about the computational side of things, but why do these approaches work at all?

In statistics and machine learning many of our problems have a form that looks like,

$$\arg\min_{\theta} \ \ell(\boldsymbol{X}, \theta) = \arg\min_{\theta} \ \frac{1}{n}\sum_{i=1}^{n} \ell(\boldsymbol{X}_i, \theta)$$

which means that the gradient of the loss function is given by

$$\nabla\ell(\boldsymbol{X}, \theta) = \frac{1}{n}\sum_{i=1}^{n} \nabla\ell(\boldsymbol{X}_i, \theta)$$

$$\nabla\ell(\boldsymbol{X}, \theta) \approx \frac{1}{|B|}\sum_{i\in B} \nabla\ell(\boldsymbol{X}_i, \theta)$$

SGD estimator

Because we are sampling B randomly, then our SGD and mini batch GD approximations are unbiased estimates of the full gradient,

[Math Processing Error]

Each update can be viewed as a noisy gradient descent step (gradient + zero mean noise).

- The difference between mini batch and stochastic gradient descent is that by increasing the computation cost per step we are reducing the noise variance for that step

Limitations

As mentioned previously we need to be a bit careful with learning rates and convergence for both of these methods. So far, our approach has been naive and runs for a fixed number of epochs.

If we want to use a convergence criterion we need to keep the following in mind:

- Let θ^* be a global / local minimizer of our loss function $\ell(\boldsymbol{X}, \theta)$, then by definition $\nabla \ell(\boldsymbol{X}, \theta^*) = 0$
- The issue is that our gradient approximation,

$$\frac{1}{|B|} \sum_{i \in B} \nabla \ell(\boldsymbol{X}_i, \theta) \neq 0$$

  as B is a subset of the data, therefore our algorithm is likely to never converge.

Solution

The practical solution to this is to implement a learning rate schedule which generally shrinks the learning rate / step size over time to ensure convergence.

The choice of the exact learning schedule is problem specific, and is usually about finding the right balance.

Some common examples:

- Piecewise constant - $\eta_t = \eta_i \text{ if } t_i \leq t \leq t_{i+1}$
- Exponential decay - $\eta_t = \eta_0 e^{-\lambda t}$
- Polynomial decay - $\eta_t = \eta_0 (\beta t + 1)^{-\alpha}$

There are many more approaches including more exotic techniques that allow the learning rate to increase and decrease to help the optimizer better explore the objective function and in some cases escape local optima.

Adaptive Updates & Momentum

AdaGrad

This approach was proposed by Duchi, Hazan, & Singer in 2011 and is based on the idea of scaling the learning rates for the current step by the sum of the square gradients of previous steps - this has the effect of shrinking the step size of dimensions with large previous gradients.

[Math Processing Error]

here ϵ is a small constant (i.e. 10^{-7}) to avoid division by zero.

Implementation

```
adagrad_lm = function(X, y, beta, step, batch_size = 10, max_step = 50, seed = 1234, replace
  X = cbind(1, X)
  n = nrow(X); k = ncol(X)
  f = \(beta) sum((y - X %*% beta)^2)
  grad_b = \(beta, idx) 2 * t(X[idx, , drop = FALSE]) %*% (X[idx, , drop = FALSE] %*% beta -

  res = list(x = list(beta), loss = f(beta), iter = 0)
  set.seed(seed)
  S = rep(0, k)

  for (ep in seq_len(max_step)) {
    if (replace) {
      js = sample.int(n, n, replace = TRUE)
    } else {
      js = sample.int(n)
    }

    batches = split(js, ceiling(seq_along(js) / batch_size))
    for (batch in batches) {
      G = drop(grad_b(beta, batch))
      S = S + G^2
      beta = beta - step * (1 / sqrt(S + eps)) * G

```

A medium example

```r
set.seed(1234)
n = 1000; p = 20
X = matrix(rnorm(n * p), n, p)
true_beta = rep(0, p)
true_beta[c(3, 7, 12, 18)] = c(5.2, -3.1, 8.7, -1.5)
y = 3 + X %*% true_beta + rnorm(n)
```

```r
lm_fit = lm(y ~ X)
coef(lm_fit)
```

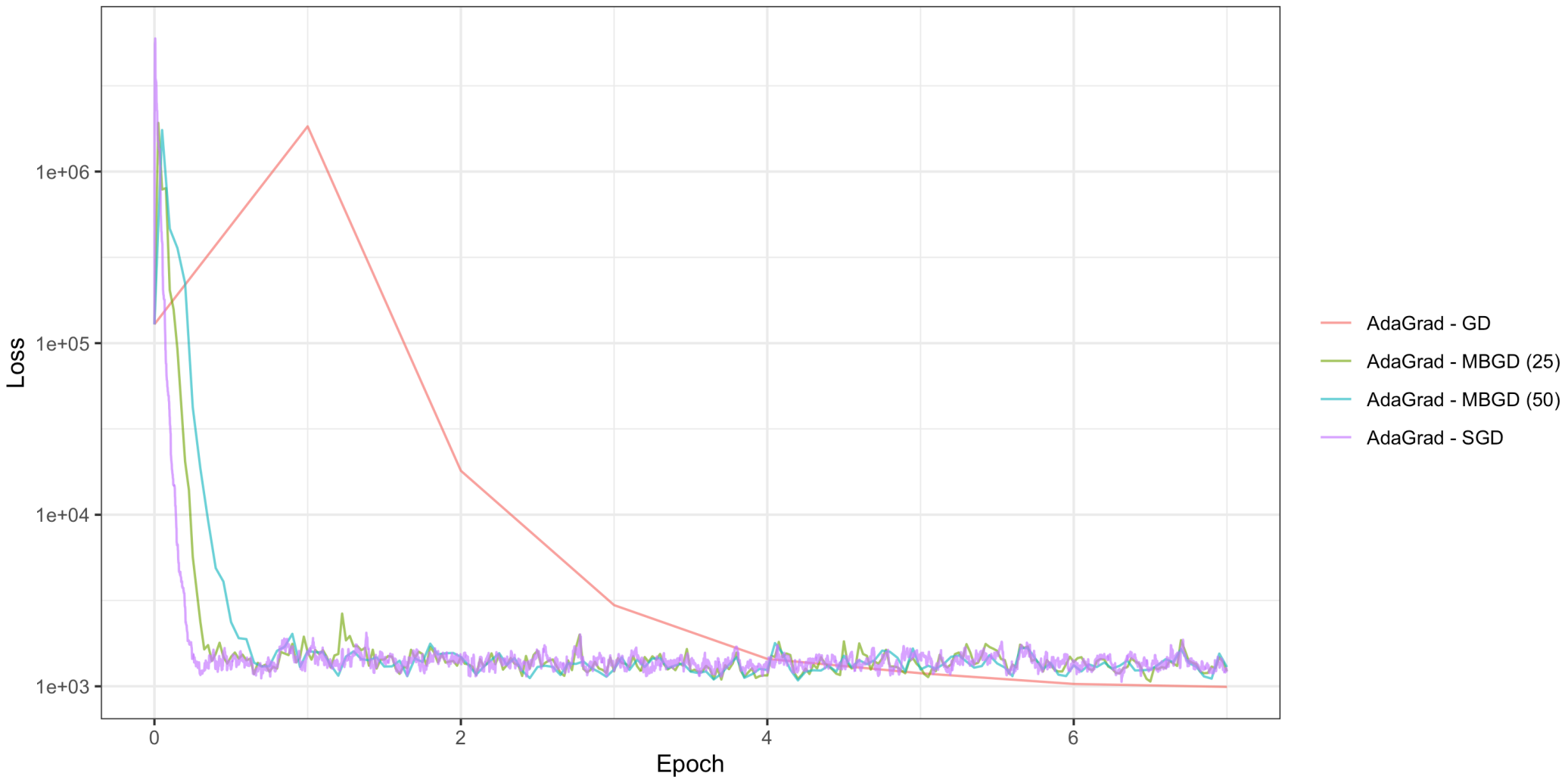
```
 (Intercept)            X1            X2            X3            X4
2.9774392131 -0.0110802711  0.0009632153  5.2627169203  0.0166936616
          X5            X6            X7            X8            X9
-0.0127002274 -0.0072890492 -3.1444361598 -0.0068887697 -0.0043521502
         X10           X11           X12           X13           X14
-0.0158086692  0.0064750332  8.7203136698 -0.0222736610 -0.0100740776
         X15           X16           X17           X18           X19
-0.0092988860 -0.0226729137 -0.0321752237 -1.4524168142  0.0325148774
         X20
-0.0071598029
```

Fitting

```r
ag_sizes = c(1, 25, 50, 1000)
ag_lrs = c(10, 10, 10, 10)
ag_algos = c("AdaGrad - SGD", "AdaGrad - MBGD (25)", "AdaGrad - MBGD (50)", "AdaGrad - GD")

adagrad_res = purrr::map2(ag_sizes, ag_lrs, \(size, lr) {
  adagrad_lm(X, y, rep(0, p + 1), batch_size = size, step = lr, max_step = 7, replace = TRUE, eps = 1e-8)
}) |> setNames(ag_algos)
```

```
AdaGrad - SGD
 [1]  2.923516954  0.077958927  0.024072555  5.328207315  0.047602717
 [6]  0.013605867 -0.042822939 -3.078479206 -0.170411448 -0.247523613
[11] -0.124192909  0.224090810  8.589651301 -0.095520815 -0.162187496
[16]  0.009713967  0.091980231 -0.134466687 -1.572626656  0.096579873
[21] -0.108601047

AdaGrad - MBGD (25)
 [1]  2.9636951336  0.0103614689  0.0006939776  5.3558159033
 [5]  0.0793669926  0.0944629851 -0.1346611885 -3.0312122629
 [9] -0.0580586918 -0.1415274039 -0.0806344971  0.1839320913
[13]  8.5693888170 -0.1972943730 -0.0719987377 -0.0450784621
[17]  0.0477064377 -0.0904873181 -1.5144441316  0.1974612861
[21] -0.2721612603

AdaGrad - MBGD (50)
 [1]  3.04863333  0.19328985  0.02224499  5.45419319  0.07342164
 [6]  0.11851767 -0.11150982 -2.80874453 -0.09346264 -0.06139525
[11] -0.07554879  0.14561157  8.50936500  0.00628026 -0.13217404
[16]  0.06647880 -0.08606309 -0.02167858 -1.56990183  0.03153194
[21] -0.14430946

AdaGrad - GD
 [1]  3.079294791 -0.011250608 -0.029757062  5.273586088  0.033510411
 [6]  0.044941529 -0.033937411 -3.204064310 -0.010172693  0.030126074
[11] -0.017119202  0.076240527  8.654543573 -0.016635521 -0.031419769
[16]  0.001304021 -0.023556850 -0.095227137 -1.502948342  0.036444505
[21] -0.044296845
```

Results


Loss
1e+06
1e+05
1e+04
1e+03
0
2
4
6
Epoch
AdaGrad - GD
AdaGrad - MBGD (25)
AdaGrad - MBGD (50)
AdaGrad - SGD

RMSProp

With AdaGrad the denominator involving $\boldsymbol{s}_t$ gets larger as t increases, but in some cases it gets too large too fast to effectively explore the loss function. An alternative is to use a moving average of the past squared gradients instead.

RMSProp replaces AdaGrad's $\boldsymbol{s}_t$ with the following,

$$\boldsymbol{s}_t = \beta\, \boldsymbol{s}_{t-1} + (1-\beta)\left(\nabla \ell(\boldsymbol{X}, \boldsymbol{\theta}_t)\right)^2$$

$$\boldsymbol{s}_0 = \mathbf{0}$$

in practice a value of $\beta \approx 0.9$ is often used.

Implementation

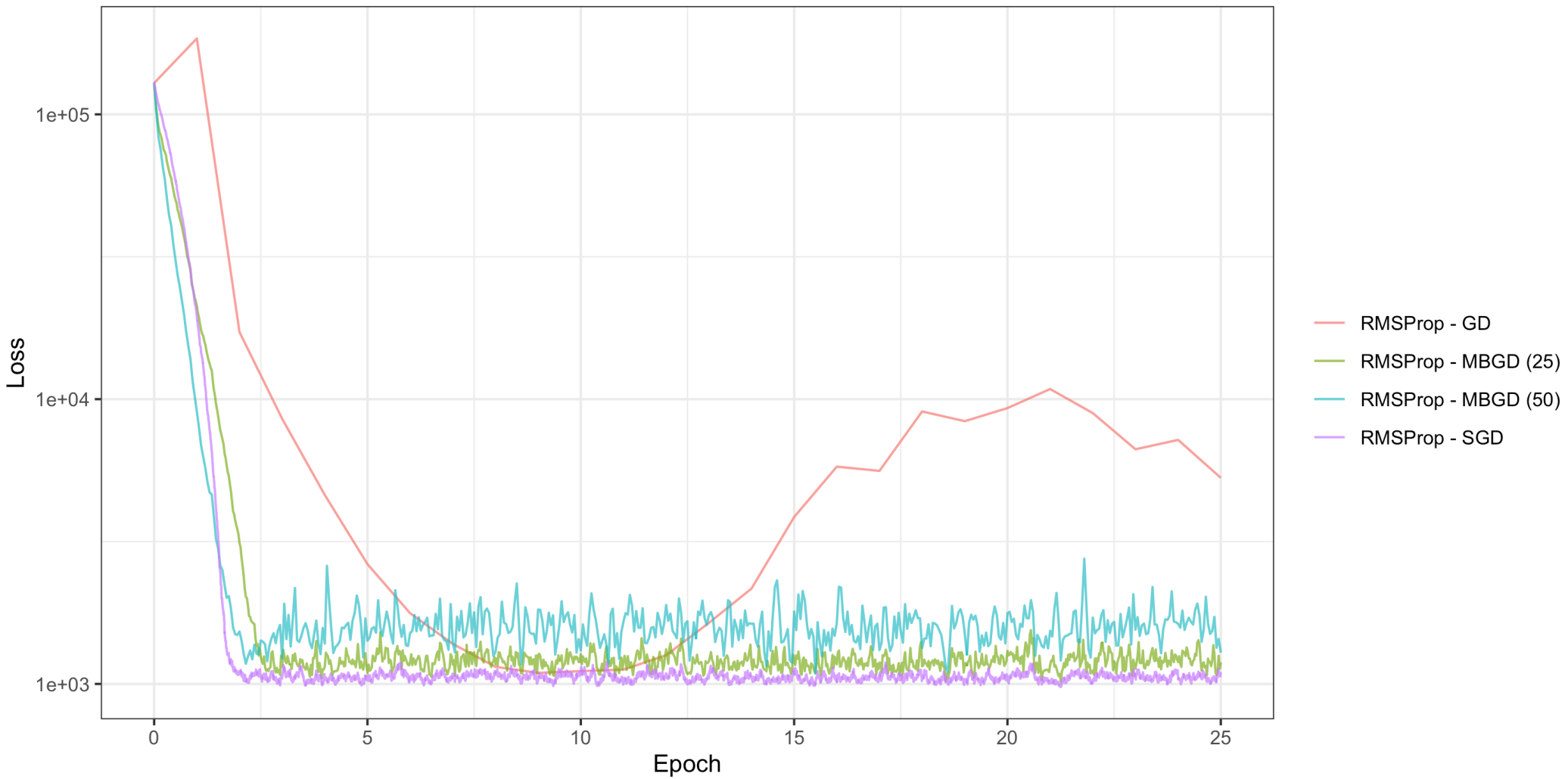
```
rmsprop_lm = function(X, y, beta, step, batch_size = 10, max_step = 50, seed = 1234, replace
  X = cbind(1, X)
  n = nrow(X); k = ncol(X)
  f = \(beta) sum((y - X %*% beta)^2)
  grad_b = \(beta, idx) 2 * t(X[idx, , drop = FALSE]) %*% (X[idx, , drop = FALSE] %*% beta -

  res = list(x = list(beta), loss = f(beta), iter = 0)
  set.seed(seed)
  S = rep(0, k)

  for (ep in seq_len(max_step)) {
    if (replace) {
      js = sample.int(n, n, replace = TRUE)
    } else {
      js = sample.int(n)
    }

    batches = split(js, ceiling(seq_along(js) / batch_size))
    for (batch in batches) {
      G = drop(grad_b(beta, batch))
      S = b * S + (1 - b) * G^2
      beta = beta - step * (1 / sqrt(S + eps)) * G

```

Fitting

```
rms_sizes = c(1, 25, 50, 1000)
rms_lrs = c(0.01, 0.1, 0.25, 1)
rms_algos = c("RMSProp - SGD", "RMSProp - MBGD (25)", "RMSProp - MBGD (50)", "RMSProp - GD")

rmsprop_res = purrr::map2(rms_sizes, rms_lrs, \(size, lr) {
  rmsprop_lm(X, y, rep(0, p + 1), batch_size = size, step = lr, max_step = 25, replace = TRUE)
}) |> setNames(rms_algos)
```

```
RMSProp - SGD
 [1]  2.92290381 -0.06205029 -0.06777944  5.17999597  0.13025550
 [6]  0.03836089  0.13041626 -2.99037012 -0.10661007  0.03472250
[11]  0.01911874  0.05360124  8.71581720 -0.05223083  0.03793280
[16] -0.06096409 -0.11420671 -0.08129282 -1.57972529  0.11667619
[21] -0.04327320

RMSProp - MBGD (25)
 [1]  2.9433199256 -0.0812563967 -0.0605549048  5.2430923281
 [5]  0.1420603391 -0.0101004209  0.1479044663 -2.9278670818
 [9] -0.0867416723  0.0343693275  0.1818238742  0.0754136148
[13]  8.7343596916 -0.0267350197 -0.0008239147 -0.0766222945
[17] -0.1303706595 -0.0549832652 -1.5775971467  0.2088404125
[21] -0.1468360467

RMSProp - MBGD (50)
 [1]  3.019476167 -0.028619061  0.004026631  5.155565226  0.054340790
 [6] -0.071360747  0.190239091 -3.027227912 -0.162446336 -0.120107757
[11]  0.175116722  0.116420900  8.791004045 -0.072378709  0.018388477
[16]  0.047496326 -0.125514052 -0.055142462 -1.638159693  0.161726611
[21] -0.328040036

RMSProp - GD
 [1]  2.61686938 -0.70016509 -0.61714751  5.41693052  0.32900712
 [6]  0.17690385 -0.12501197 -3.33654523 -0.64783728  0.61744367
[11]  0.20139679  0.07777429  8.55781405  0.60805860 -0.27913545
[16]  0.45151670  0.59738928  0.04523175 -1.98838489  0.47266779
[21]  0.13167226
```

Results


Loss
1e+05
1e+04
1e+03
0
5
10
15
20
25
Epoch
RMSProp - GD
RMSProp - MBGD (25)
RMSProp - MBGD (50)
RMSProp - SGD

Momentum

Rather than just using the gradient information at our current location it may be beneficial to use information from our previous steps as well. A general setup for this type approach looks like,

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta\, \boldsymbol{m}_t$$

$$\boldsymbol{m}_t = \beta\, \boldsymbol{m}_{t-1} + (1-\beta)\, \nabla \ell(\boldsymbol{X}, \boldsymbol{\theta}_t)$$

where η is our step size and β determines the weighting of the current gradient and the previous gradients.

If you have taken a course on time series, this has a flavor that looks a lot like moving average models,

$$\boldsymbol{m}_t = (1-\beta)\, \nabla \ell(\boldsymbol{X}, \boldsymbol{\theta}_t) + \beta(1-\beta)\, \nabla \ell(\boldsymbol{X}, \boldsymbol{\theta}_{t-1}) + \beta^2(1-\beta)\, \nabla \ell(\boldsymbol{X}, \boldsymbol{\theta}_{t-2}) + \cdots$$

Adam

The “adaptive moment estimation” algorithm is a combination of momentum with RMSProp,

[Math Processing Error]

Note that RMSProp is a special case of Adam when $\beta_1 = 0$.

Adam is widely used in practice and is commonly available within tools like Torch for fitting neural network models.

In typical use $\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-6}$, and $\eta_t = 0.001$ are used. As the learning rate is not guaranteed to decrease over time, the algorithm is not guaranteed to converge.

Bias corrections

One small alteration that was suggested by the original authors and is commonly used is to correct for the bias towards small values in the initial estimates of $\boldsymbol{m}_t$ and $\boldsymbol{s}_t$. In which case they are replaced with,

[Math Processing Error]

Implementation

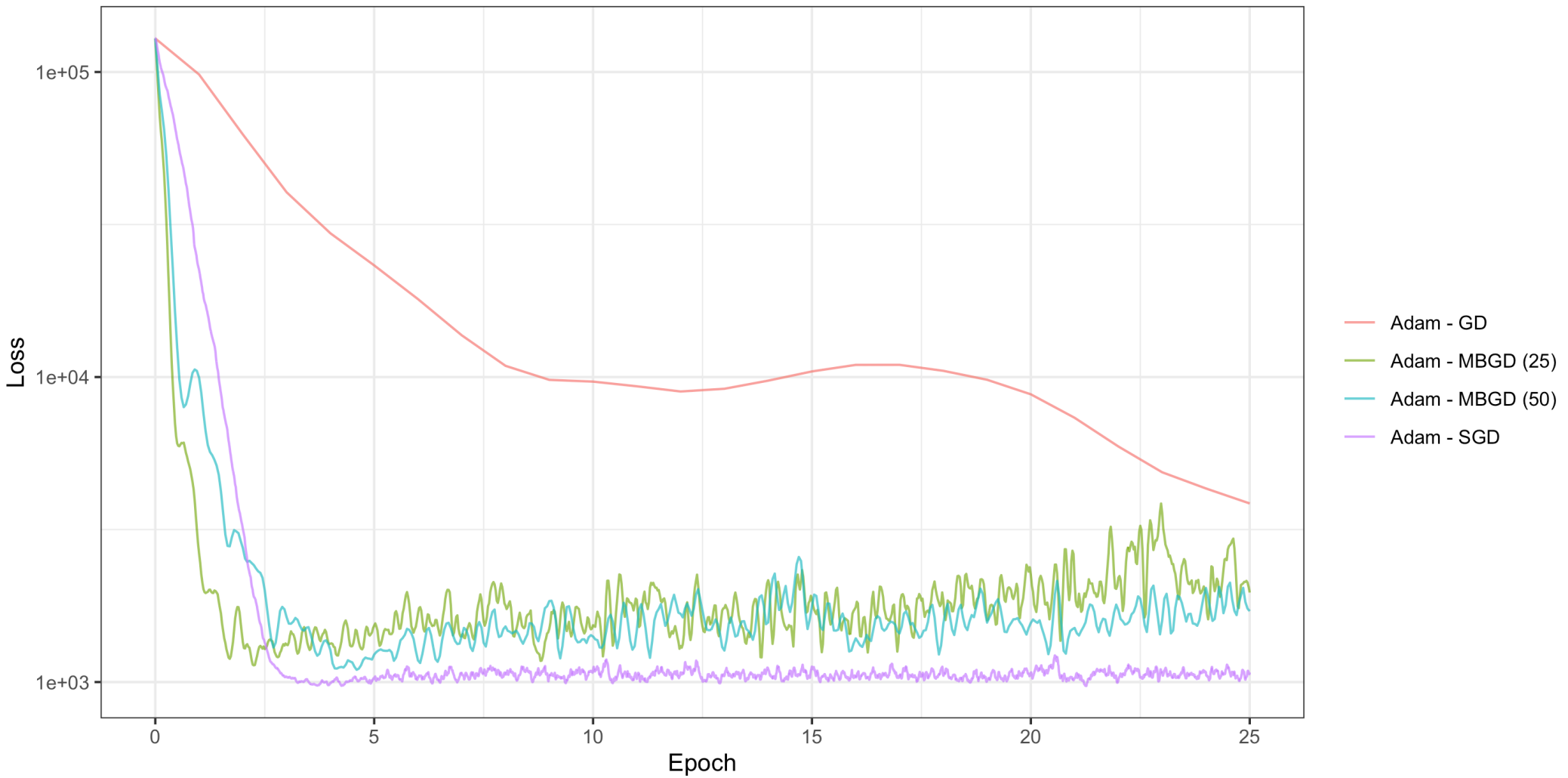
```
adam_lm = function(X, y, beta, step = 0.001, batch_size = 10, max_step = 50, seed = 1234, rep
  X = cbind(1, X)
  n = nrow(X); k = ncol(X)
  f = \(beta) sum((y - X %*% beta)^2)
  grad_b = \(beta, idx) 2 * t(X[idx, , drop = FALSE]) %*% (X[idx, , drop = FALSE] %*% beta -

  res = list(x = list(beta), loss = f(beta), iter = 0)
  set.seed(seed)

  S = rep(0, k)
  M = rep(0, k)
  t = 0

  for (ep in seq_len(max_step)) {
    if (replace) {
      js = sample.int(n, n, replace = TRUE)
    } else {
      js = sample.int(n)
    }

    batches = split(js, ceiling(seq_along(js) / batch_size))
    for (batch in batches) {
      t = t + 1
```

Fitting

```r
adam_sizes = c(1, 25, 50, 1000)
adam_lrs = c(0.01, 0.5, 0.75, 1)
adam_algos = c("Adam - SGD", "Adam - MBGD (25)", "Adam - MBGD (50)", "Adam - GD")

adam_res = purrr::map2(adam_sizes, adam_lrs, \(size, lr) {
  adam_lm(X, y, rep(0, p + 1), batch_size = size, step = lr, max_step = 25, replace = TRUE)
}) |> setNames(adam_algos)
```

```
Adam - SGD
 [1]  2.91679509 -0.10361337 -0.04001301  5.18074137  0.15335836
 [6]  0.03525088  0.07970582 -2.99598116 -0.10345424  0.02683289
[11]  0.03914728  0.07648587  8.72039585 -0.05162264  0.03692920
[16] -0.04277340 -0.09597054 -0.08488274 -1.54878316  0.11974005
[21] -0.03391827

Adam - MBGD (25)
 [1]  2.870292611 -0.433872958  0.006486318  5.207245654  0.264780478
 [6]  0.116681715  0.193177568 -2.616395118  0.045210581  0.226068423
[11] -0.314439706  0.041536235  8.944863320 -0.347555372  0.071351484
[16]  0.142292428  0.124914322 -0.264111292 -1.392145981 -0.090874110
[21]  0.235883567

Adam - MBGD (50)
 [1]  2.70982296 -0.12345292 -0.08366327  5.06722373  0.07011974
 [6] -0.13016125  0.07216297 -3.03532359 -0.22403459  0.01333613
[11]  0.13760969  0.23568917  8.65956807  0.08283943 -0.02883813
[16] -0.09148560 -0.06527121 -0.24089260 -1.40634619  0.59085108
[21] -0.24137533

Adam - GD
 [1]  3.017049628  0.221629092 -0.301279041  4.305498734 -0.007810589
 [6]  0.057462297  0.122413673 -3.647735491 -0.159905985  0.216647445
[11] -0.358532627  0.081423712  9.707972439 -0.033692214 -0.091597262
[16]  0.183828857  0.500689450 -0.204335822 -1.398305519 -0.134240531
[21]  0.362516615
```

Results


1e+05
1e+04
1e+03
Loss
0
5
10
15
20
25
Epoch
Adam - GD
Adam - MBGD (25)
Adam - MBGD (50)
Adam - SGD